Accelerating Suffix Jailbreak attacks with Prefix-Shared KV-cache

Xinhai Wang*, Shaopeng Fu*, Liangyu Wang, Shu Yang, and Di Wang
King Abdullah University of Science and Technology
Thuwal, Saudi Arabia
{xinhai.wang,shaopeng.fu,liangyu.wang,shu.yang,di.wang}@kaust.edu.sa

Abstract

While large language models (LLMs) have been widely deployed and applicable to real-world scenarios, their security is found to be vulnerable under various attacks. A major family of methods for assessing LLMs' security is *suffix jailbreak attacks*, a powerful class of attacks that optimize adversarial suffixes to circumvent safety alignment and induce harmful behaviors from LLMs. However, launching suffix attacks is resource-intensive because finding effective jailbreak suffixes requires solving sophisticated discrete optimization problems.

This paper presents Prefix-Shared KV cache(PSKV), which can significantly improve the efficiency of existing suffix jailbreak attacks by reducing both *memory* and *computation* costs during the suffix generation process. We observe that when synthesizing a single jailbreak suffix, while a large number of candidate prompts need to be evaluated by the targeted LLM for jailbreak evaluation, these prompts share the same prefix, *i.e.*, the targeted harmful instruction. Built upon this key observation, PSKV only maintains a single KV-cache for every single harmful instruction and uses it to support the inference of multiple candidate jailbreak prompts, which thus reduces the memory cost by several times. Besides, such a KV-cache-sharing design enables us to extend PSKV to calculate jailbreak suffixes for a batch of different inputs in parallel with little memory overhead, thus one can further reduce the computation cost via efficient batching operations. We evaluate six common suffix attacks on five widely deployed LLMs using 50 samples from the HarmBench dataset. Results demonstrate that PSKV reduces time and memory costs by approximately 40% and 30%, respectively, providing substantial acceleration for suffix attacks while preserving attack effectiveness.

CCS Concepts

• **Security and privacy** → **Systems security**; • **Computing methodologies** → **Artificial intelligence**; *Natural language processing*; Machine learning algorithms.

Keywords

Jailbreak Attack,LLM Safety,KV Cache

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

ACM Reference Format:
Xinhai Wang*, Shaopeng Fu*, Liangyu Wang, Shu Yang, and Di Wang. 2025. Accelerating Suffix Jailbreak attacks with Prefix-Shared KV-cache. In . ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

While large language models (LLMs) have been widely deployed in real-world applications, they are also found to be facing several security challenges [4, 5, 22]. One of the most serious threats is jailbreak attacks [20], which aim to induce arbitrary harmful behaviors from targeted LLMs with specifically crafted *jailbreak prompts*. To better understand this security issue, various jailbreak attacks have been used for automated red-teaming of LLM robustness [1, 15]. Among them, a major family of attacks is *suffix (jailbreak) attacks*, which bypasses model safety alignment by appending an automatically optimized adversarial suffix to a harmful instruction [8, 11, 16, 18, 25, 26]. Due to its input-level nature and high effectiveness, suffix attacks have been widely adopted in benchmarking the robustness of LLMs against jailbreaking attacks. [12–14].

A standard suffix jailbreak attack can be formalized as solving a discrete optimization problem that searches for jailbreak suffixes in the discrete LLM token space, with the goal of maximizing the probability of LLMs generating targeted harmful content [18, 26]. Such a process usually generates a vast number of candidate suffixes, and targeted LLMs need to perform inference on these candidate suffixes to assess their jailbreak effectiveness, which requires a lot of GPU resources and time. To improve the efficiency of the standard suffix jailbreaking, recent studies suggest using LLM-based jailbreak suffix generators to directly generate jailbreak suffixes for given harmful instructions [6, 11, 16]. Unfortunately, such a solution only moves the suffix synthesis cost from "test-time" to "training-time", as one still needs to find a set of jailbreak suffixes by solving the aforementioned costly discrete optimization problem to train the jailbreak suffix generator. All these technical challenges create prohibitive costs for large-scale LLMs security evaluations [16, 26].

To enable more efficient LLM jailbreak robustness evaluation, this paper presents ***Prefix-Shared KV Cache (PSKV)***, a method that can significantly accelerate existing suffix jailbreak attack methods via reducing both their *memory* and *computation* costs. Our solution stems from the key observation that when searching for jailbreak suffixes in the discrete LLM token space, while a large number of candidate jailbreak prompts need to be fed into the targeted LLM, all these candidate prompts share the same prefix, *i.e.*, the original harmful instruction. As (candidate) jailbreak suffixes are usually much shorter than the corresponding harmful instruction prefix, if the LLM inference result of such a prefix can be calculated and stored only once, then the memory usage and computational cost during the inference process in suffix jailbreaking are expected to be significantly reduced.

Built upon such an observation, when using PSKV to accelerate suffix jailbreaking, for a single harmful instruction, our approach only maintains a single piece of key-value (KV) cache [3, 10] for it and uses this single KV cache to support the inference of multiple candidate jailbreak prompts. Besides, PSKV introduces a suffix-centric alignment strategy that enables efficient batched operations throughout the suffix generation process. Unlike conventional left-alignment approaches that require extracting individual suffixes and targets for computation, our suffix-centric alignment allows direct batch-wise loss calculation, suffix updating, and gradient computation on aligned tensors. The reduced memory footprint from KV cache sharing creates headroom for larger batch sizes, which combined with the efficient batched operations enabled by suffix-centric alignment, leads to substantial acceleration in the overall suffix attack process.

We empirically justify the effectiveness of our PSKV approach on six popular suffix jailbreak attacks against five widely deployed LLMs using 50 samples from the HarmBench datase. Experimental results show that compared with original implementations, PSKV reduces suffix synthesis time by an average of 40% and peak memory consumption by 30% compared with vanilla KV cache-optimized implementations. Since KV cache is a lossless optimization that preserves LLM inference results, PSKV maintains the original attack effectiveness while achieving these speedups. Additionally, we present a comprehensive analysis of trade-offs between memory savings and computational overhead, offering guidance for optimal deployment in resource-constrained environments. Our work paves the way for more efficient and scalable LLM security evaluation.

2 Related Work

In this section, we first survey the landscape of adversarial attack methodologies and then discuss relevant advancements in efficient LLM inference, highlighting the specific computational niche our work addresses.

2.1 Suffix Jailbreak Methods for LLMs

Research in automated jailbreaking has produced a diverse set of methodologies for generating adversarial suffixes. These can be broadly categorized into search/optimization-based and LLM-based approaches.

Search and Optimization-based Attacks. This line of work treats suffix generation as a search problem. The foundational method, GCG [26], employs a greedy, gradient-guided search to iteratively refine an adversarial suffix. Building on this, subsequent works have explored more sophisticated search strategies. BEAST [18] replaces the greedy algorithm with beam search to explore a wider range of promising candidates. Zhu's AutoDAN [25] utilizes genetic algorithms to evolve suffixes, mitigating the local optima problem of purely greedy methods. Other variants like GCQ (Gradient-based Attack with Quantization) [8] focus on improving the efficiency of the core gradient-based update step itself. While these methods are powerful, their reliance on extensive, iterative search results in significant computational overhead.

LLM-based Attacks. An alternative paradigm uses LLMs themselves as generators. AdvPrompter [16] fine-tunes a language model to automatically compose adversarial prompts. Similarly, AmpleGCG trains a generative model to directly output effective suffixes, amortizing the search cost into a one-time training process [11]. Despite their different approaches, these methods still face a major computational bottleneck during their data collection phase, which requires evaluating a massive number of candidate suffixes to create a training set. Our work presents an orthogonal optimization that accelerates this critical evaluation step, benefiting both categories of attack.

2.2 Efficient LLM Inference and Caching

The KV cache is a fundamental technique for accelerating autoregressive inference by reusing the computations for previous tokens [17]. However, the memory footprint of the KV cache itself is a primary bottleneck. This has spurred innovations in general-purpose inference optimization. The most prominent of these is PagedAttention, the core technology of the vLLM system [10], which manages the KV cache in non-contiguous memory pages to mitigate fragmentation and improve throughput in multi-user serving scenarios.

Recent work has explored automatic KV cache reuse through prefix-aware structures. SGLang [24] introduces RadixAttention, which organizes KV cache in a radix tree to automatically detect and share common prompt prefixes across requests at runtime, eliminating the need for manual configuration. Building on similar observations about shared prefixes, ChunkAttention [23] proposes a two-phase partition algorithm that chunks key/value tensors and employs chunk-first and sequence-first phases to optimize memory locality during self-attention computation.

While these general-purpose optimizations are powerful, they are not designed for the unique memory access patterns of adversarial suffix generation, where a long, static prefix is repeatedly processed with minor suffix variations. This massive data redundancy is not explicitly exploited by systems like vLLM. Our work, in contrast, introduces a workload-specific caching strategy that leverages this redundancy, rather than focusing on a general-purpose serving solution.

3 Preliminaries

Large language models (LLMs). An attention-based LLM can be seen as a parameterized distribution p_θ that is defined over a discrete vocabulary $[V] = \{1, \ldots, V\}$. Given an input prompt $x_{1:n} \in [V]^n$ that consists of n tokens, the LLM p_θ will generate a response $y_{1:m} \in [V]^m$ of length m via sampling it from the conditional distribution $p_\theta(y_{1:m}|x_{1:n}) = \prod_{i=1}^{m} p_\theta(y_i|x_{1:n} \oplus y_{1:(i-1)})$, where $\oplus$ denotes the concatenation operator.

Suffix jailbreak attacks. Given a harmful prompt $x^{(h)}$ (*e.g.*, "How to build a bomb") and a corresponding desired malicious response(i.e., the targeted text) $y^{(h)}$ (*e.g.*, "Sure, here are the instructions..."), a suffix jailbreak attack aims to synthesize a jailbreak suffix $x^{(s)} \in [V]^m$ of a given length m to maximize the probability of generating the response $y^{(h)}$ when concatenating the suffix with the original prompt $x^{(h)}$. Formally, the synthesis of the adversarial suffix $x^{(s)}$ can be

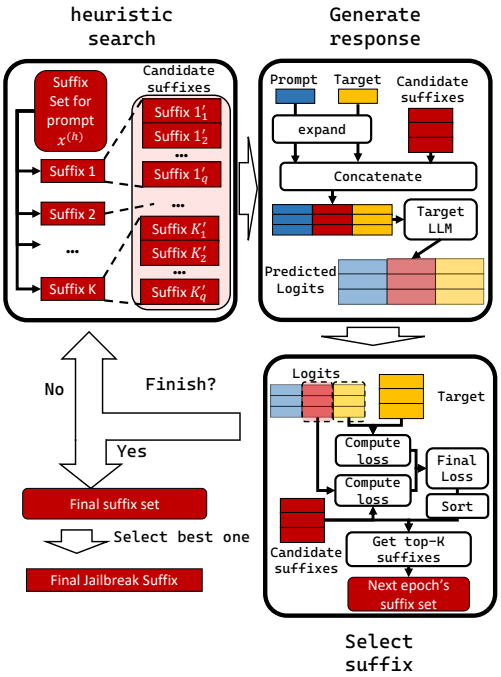

Figure 1: General adversarial suffix generation processes. "Target" is the abbreviation of "Targeted Text".

defined as the following optimization problem,

$$\min_{x_{1:m}^{(s)} \in [V]^m} -\log p_\theta(y^{(h)} | x^{(h)} \oplus x_{1:m}^{(s)}). \tag{1}$$

Solving Eq. (1) usually requires leveraging heuristic methods that iteratively propose new *candidate* adversarial suffixes based on suffixes found in previous iterations. Specifically, suppose in each iteration of the jailbreak suffix synthesis, $K \in \mathbb{N}^+$ synthesized suffixes will be kept as candidate jailbreak suffixes. Then, the general process of generating candidate suffixes in each iteration can be formalized in the following three steps:

- **Step 1:** For each suffix in the previous iteration's suffix set, propose q new suffixes via heuristic search methods. This results in $K \cdot q$ candidate suffixes generated in this round.
- **Step 2:** Expand the harmful prompt $x^{(h)}$ and the targeted response $y^{(h)}$ for $K \cdot q$ times to match the number of candidate suffixes, concatenate each generated suffix with its corresponding expanded prompts and responses. Then use the LLM p_θ to perform inference on these concatenated inputs to get the predictions for the targeted texts. This results in up to $K \cdot q$ LLM inferences in this round.
- **Step 3:** For each of the $K \cdot q$ candidates, calculate its loss using the model's output from **Step 2**, the targeted response $y^{(h)}$ and candidate suffixes. Then, select the K suffixes with the lowest loss values as the jailbreak suffix set for this iteration.

The final synthesized jailbreak suffix is chosen from the K suffixes in the last iteration. The general process of jailbreak suffix synthesis is also illustrated in Fig. 1.

Model-based Suffix Generation. Distinct from the direct optimization of the adversarial suffix, an alternative paradigm involves training an auxiliary model, termed a jailbreak prompter, to automatically generate effective suffixes. The objective thus shifts from finding an optimal suffix $x^{(s)}$ for a fixed target model p_θ to optimizing the parameters ϕ of a prompter model g_ϕ that learns the distribution of effective jailbreak suffixes.

To achieve this, we first curate a dataset of successful jailbreak pairs $\mathcal{D}$. The creation of this dataset involves a two-stage process: candidate generation and success-based filtering. Initially, for each harmful prompt $x_i^{(h)}$ in a given set, a collection of candidate suffixes $\mathcal{S}_{\text{cand},i}$ is pre-generated using the heuristic methods described previously. Subsequently, each candidate suffix $s \in \mathcal{S}_{\text{cand},i}$ is filtered based on its success. An attack is deemed successful if the output from the target LLM p_θ does not contain any keywords from a manually curated set of refusal terms $\mathcal{R}$ (*e.g.*, "I cannot", "unable to"). Formally, letting $W(y)$ be the set of words in an output string y, the final dataset $\mathcal{D}$ is constructed by only including pairs that satisfy the following success condition:

$$y_s = \operatorname*{argmax}_y p_\theta(y | x_i^{(h)} \oplus s), \tag{2}$$

where $\mathcal{D} = \{(x_i^{(h)}, s) \mid s \in \mathcal{S}_{\text{cand},i} \text{ and } W(y_s) \cap \mathcal{R} = \emptyset\}$. The goal is then to train the prompter model g_ϕ on this curated dataset $\mathcal{D}$ to generate effective suffixes. This can be formulated as a standard conditional language modeling objective:

$$\min_\phi \mathbb{E}_{(x^{(h)}, x^{(s)}) \sim \mathcal{D}} [-\log p_{g_\phi}(x^{(s)} | x^{(h)})]. \tag{3}$$

The process for model-based suffix generation can be formalized in the following steps:

- **Step 1: Jailbreak Data Curation.** For a set of harmful prompts $\{x_i^{(h)}\}$, employ the iterative heuristic methods to synthesize a corresponding set of candidate adversarial suffixes. Then filter these candidates, retaining only the successful pairs $(x_i^{(h)}, x_i^{(s)})$ where the target LLM's response does not contain refusal keywords, as defined in Eq. (2). This creates the training dataset $\mathcal{D}$.
- **Step 2: Prompter Model Training.** An auxiliary language model, the jailbreak prompter g_ϕ, is then fine-tuned on the curated dataset $\mathcal{D}$. The training objective is to minimize the negative log-likelihood of generating the target suffix given the harmful prompt, as defined in Eq. (3).
- **Step 3: Suffix Inference.** Once trained, the prompter g_ϕ can generate a new jailbreak suffix $x_{\text{new}}^{(s)}$ for any given harmful prompt $x^{(h)}$ in a single forward pass: $x_{\text{new}}^{(s)} \sim g_\phi(\cdot | x^{(h)})$.

KV cache. When an attention-based LLM p_θ generates the i-th token of a sequence, it needs to refer to the key and value (KV) vectors (i.e., vectors projected by key matrix and value matrix in every attention module in the LLM) of all the previous $(i-1)$ tokens. To avoid redundant KV vector calculations, *KV cache* will store all these KV vectors once they are generated, so that they can be used in subsequent generation processes. While KV cache can significantly reduce the calculation and time costs of LLM inference, when the LLM p_θ is generating a batch of sequences, the KV cache

needs to store KV vectors for each sequence within the batch, which might cause significant memory overhead.

4 Methodology

In this section, we first illustrate the redundant calculation problem of prefix KV vectors in suffix jailbreaking, and then introduce our *PSKV* method to tackle the raised challenge and significantly improve the efficiency of suffix jailbreak attacks.

4.1 Redundant Prefix Calculations in Suffix Jailbreak Attacks

According to the general jailbreak suffix synthesis process described in Section 3, the main calculation burden of suffix jailbreaking comes from performing the $K \cdot q$ times of LLM inference (in **Step 2**) in each synthesizing iteration, where K is the number of candidate suffixes in each iteration and q is the number of new suffixes generated for each candidate. Larger K and q usually make the jailbreak attacks explore more candidate suffixes and thus have a better chance to find an effective jailbreak suffix, but also lead to more LLM inference times.

Fortunately, we observe that while the number of candidate prompts that an LLM needs to perform inference on in each jailbreaking iteration is as large as $K \cdot q$, these prompts all share the same prefix, *i.e.*, the original harmful instruction $x^{(h)}$. Specifically, as illustrated in Fig. 1, the suffix jailbreak attack only modifies the suffixes of these candidate prompts, but keeps the harmful instruction $x^{(h)}$ as their prefix intact. As a result, when an LLM is performing inference on these prompts in parallel, it will redundantly calculate the same KV vectors for the same prefix $x^{(h)}$ for up to $K \cdot q$ times. Furthermore, as shown in Table. 1, the average token length of instructions in HarmBench is 78.4, which is significantly larger than the usual jailbreak suffix length in practice (which is usually set to 20). All these observations suggest that if the KV vectors of the prefix harmful instruction $x^{(h)}$ can be calculated only once and be used for the inference of different candidate prompts, then both the *computation* and *memory* costs of suffix jailbreaking can be significantly reduced since a large amount of redundant intermediate KV vectors now do not need to be calculated.

These observations highlight a clear opportunity for optimization. The most straightforward implementation, which we term the **traditional attack method without acceleration**, treats each of the $K \cdot q$ candidate prompts as independent inputs in every iteration. This approach repeatedly re-computes the KV vectors for the prefix $x^{(h)}$, leading to significant computational overhead.

A more advanced baseline would employ a **standard KV cache** implementation. The KV vectors for the prefix $x^{(h)}$ are computed only once before the iterative synthesis process begins. However, to evaluate all $K \cdot q$ suffix candidates in a single parallel batch, this pre-computed prefix cache must be duplicated $K \cdot q$ times to match the batch dimension of the suffixes, introduces a severe memory bottleneck. For a batch of B prompts, this problem is exacerbated, requiring $B \cdot K \cdot q$ copies of the caches.

4.2 PSKV for Single Jailbreak Suffixes Synthesis

Based on our previous analysis, we now propose *PSKV*, a method that can significantly accelerate existing suffix jailbreak attacks by reducing redundant calculations of prefix KV vectors during the candidate prompts inference process of suffix jailbreaking.

Table 1: The average length (in characters) of harmful prompts and targeted texts in HarmBench.

Dataset	Harmful Prompt	Targeted Text
HarmBench	78.4	93.3

The overall procedures are also illustrated in Figure 2. Specifically, before a suffix jailbreak attack starts the iterative suffix searching process in Figure. 1, PSKV runs a single forward pass for the harmful instruction $x^{(h)}$ to calculate and cache its KV vectors in all attention layers of the targeted LLM. Then, when the targeted LLM is performing inference on the $K \cdot q$ candidate prompts during the iterative suffix searching process, whenever the LLM needs to use the KV vectors of the prefix from an attention layer, PSKV will directly take the corresponding KV vectors from its cache and duplicate these vectors $K \cdot q$ times for the LLM to support its inference process. Compared with that in vanilla suffix jailbreaking, our PSKV method reduces the time-consuming prefix KV cache calculations from $K \cdot q$ times to only a single time, which therefore can significantly improve the calculation efficiency of suffix jailbreaking.

Furthermore, we note that the KV vector duplicating procedure in PSKV requires little memory overhead. This is because during LLM inference, the outputs of all LLM attention layers are calculated in a layer-by-layer manner, and the calculation process only requires the prefix KV vectors from that specific layer. As a result, the prefix KV vector duplicating procedure in PSKV is also performed in a layer-by-layer manner, and at any one time, only KV vectors from a single layer are duplicated, which thus requires only a small amount of memory when compared with KV vectors of all layers that are redundantly repeated in vanilla suffix jailbreaking.

Complexity analysis. Let E be the number of synthesis iterations; N_p, N_s, N_t the token lengths of the prefix, suffix, and target; and $K \cdot q$ the candidates per iteration. We compare the total computational (C) and memory (M) complexities. We provide the detailed proof in Appendix B.1.

A **method without acceleration** that recomputes each sequence has a computational complexity of $C_{\text{NoAccel}} = O(E \cdot (K \cdot q) \cdot (N_p + N_s + N_t)^2)$ and a peak memory footprint of $M_{\text{NoAccel}} = O((K \cdot q) \cdot (N_p + N_s + N_t))$.

Standard KV cache improves this by pre-computing the prefix, yielding a theoretical computational cost of $C_{\text{Std}} = O(N_p^2 + E \cdot (K \cdot q) \cdot (N_s + N_t)(2N_p + N_s + N_t))$. Achieving this in a parallel batch requires physically duplicating the prefix cache, leading to a high memory footprint of $M_{\text{Std}} = O((K \cdot q) \cdot (N_p + N_s + N_t))$.

Our **PSKV** achieves the same computational complexity $C_{\text{Ours}} = C_{\text{Std}}$ with a lower memory footprint of $M_{\text{Ours}} = O(N_p + (K \cdot q) \cdot (N_s + N_t))$ by lowering the prefix cache memory term from $O((K \cdot q) \cdot N_p)$ to $O(N_p)$, which enables a much larger search width ($K \cdot q$) on the same hardware.

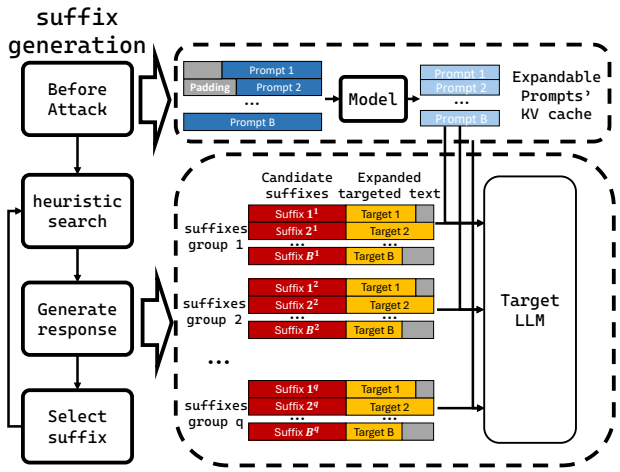

Figure 2: PSKV. By reusing the KV cache computed from the prefix, we only need to store one copy of the prompt for q new suffixes, rather than q copies as required by traditional methods.

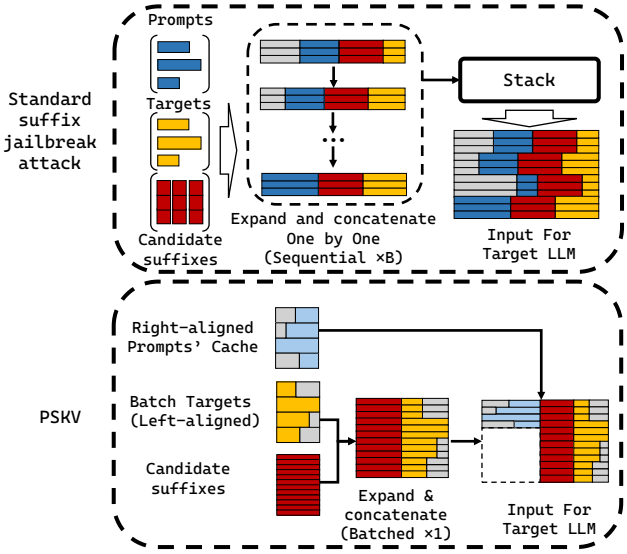

Figure 3: Alignment strategies for batched suffix attacks. The standard method's misaligned structure prevents prompt KV cache reuse. PSKV utilizes a suffix-centric alignment, enables the prompt's KV cache to be shared across all candidates and allows all components to be processed in a single, highly efficient tensor operation.

4.3 PSKV for Batches of Jailbreak Suffixes Synthesis

We now turn to analyze a more common and important scenario in real-world LLM jailbreak robustness benchmarking where one needs to synthesize jailbreak suffixes for a *batch* of harmful instructions. Specifically, suppose one needs to synthesize jailbreak suffixes of length N_s for a batch of B different harmful instruction-response pairs $\{(x_i^{(h)}, y_i^{(h)})\}_{i=1}^{B}$. Then, the main technical challenge lies in how to enable PSKV to leverage efficient tensor operations provided by deep learning frameworks such as PyTorch and TensorFlow so that the suffix calculation process for different harmful instructions can be accelerated in parallel.

To tackle the challenge, we notice that while different harmful instructions and responses may have different token lengths, in each iterative searching process during suffix jailbreaking, the generated candidate jailbreak suffixes share the same suffix length $l \leq N_s$. Therefore, we propose to: (1) construct a **right-aligned instructions token batch** by left-padding every harmful instruction $x_i^{(h)}$ so that all their right ends are aligned to the same token position within the batch, (2) construct a **left-aligned responses token batch** by right-padding every harmful response $y_i^{(h)}$ so that all their left beginnings are aligned to the same token position within the batch, and (3) pre-calculate and cache KV vectors for the right-aligned batched instructions constructed in (1). Then, as shown in Figure 3, when the targeted LLM is performing inference on candidate prompts in a searching iteration of suffix jailbreaking, while there are up to $B \cdot K \cdot q$ different candidate prompts, all candidate suffixes are formulated as a single $(BKq) \times l$ token matrix with the help of the previously constructed right-aligned and left-aligned token batches.

Such a design provides two key advantages for our PSKV method. Firstly, it enables PSKV to again use the KV cache duplicating procedure described in Section 4.2 to support low computation and memory costs LLM inference during suffix jailbreaking with the pre-calculated KV vectors for the right-aligned instructions batch. Secondly, since candidate suffixes in each searching iteration are now formalized as a $(BKq) \times l$ matrix, PSKV can easily leverage fast tensor operations from common deep learning frameworks to efficiently craft a new candidate suffixes matrix for the next iteration from the old one.

Complexity analysis. For a batch of B prompts over E iterations, let $N_{p,\max}$ and $N_{t,\max}$ be the maximum aligned lengths, N_s the suffix length, and $N_{p,\text{avg}}$, $N_{t,\text{avg}}$ be the average lengths. The total number of candidates is $B \cdot K \cdot q$. We provide the detailed proof in Appendix B.2.

A **method without acceleration** that recomputes the entire sequence for each candidate has a prohibitive computational complexity of $C_{\text{NoAccel}} = O(E \cdot (B \cdot K \cdot q) \cdot (N_{p,\text{avg}} + N_s + N_{t,\text{avg}})^2)$ and a peak memory footprint of $M_{\text{NoAccel}} = O((B \cdot K \cdot q) \cdot (N_{p,\text{avg}} + N_s + N_{t,\text{avg}}))$.

A **standard KV cache** implementation improves the computation to $C_{\text{Std}} = O(B \cdot N_{p,\max}^2 + E \cdot (B \cdot K \cdot q) \cdot (N_s + N_{t,\max})(2N_{p,\max} + N_s + N_{t,\max}))$, but its memory footprint remains high at $M_{\text{Std}} = O((B \cdot K \cdot q) \cdot (N_{p,\max} + N_s + N_{t,\max}))$ due to cache duplication.

Our framework's **PSKV** achieves the same efficient computational complexity $C_{\text{Ours}} = C_{\text{Std}}$, but significantly reduces the memory footprint to $M_{\text{Ours}} = O(B \cdot N_{p,\max} + (B \cdot K \cdot q) \cdot (N_s + N_{t,\max}))$. The key advantage is that the prefix cache storage term, $O(B \cdot N_{p,\max})$, scales with the number of prompts B, not the total candidates $B \cdot K \cdot q$, making large-scale parallel attacks practical.

5 Experiments

In this section, we empirically explore how our proposed PSKV method can reduce calculation and memory costs of suffix jailbreakings while preserving their attack effectiveness.

5.1 Experimental Setup

Safety dataset. We use the first 50 samples from the test set of **Harmbench** [14] to form the safety dataset for the suffix jailbreaking evaluations. Each sample from this safety dataset consists of a harmful instruction and a targeted harmful response. The goal of suffix jailbreaking is to synthesize a jailbreak suffix for each harmful instruction to induce the targeted LLM to generate the corresponding targeted response.

Targeted Models. We conduct suffix jailbreak attacks against five common open-source LLMs, which are: **Vicuna-7B-v1.5** [2], **Llama-2-7B-Chat** [19], **Llama-3-8B-Instruct** [7], **Mistral-7B-Instruct-v0.3** [9], and **Qwen2.5-7B-Instruct** [21]. All these models can be downloaded from the Hugging Face Model Repository [1].

Suffix jailbreak attacks. We investigate how our PSKV can improve the efficiency of conducting six common suffix jailbreak attacks from two families. Specifically, the first four attacks are from the family of optimization-based attacks, which aim to search for jailbreak suffixes with various discrete optimization techniques. They are: **GCG** [26], **GCQ** [8], **Zhu's AutoDAN** [25], and **BEAST** [18]. Besides, the last two attacks are from the family of model-based attacks, which aim to train LLM-based generators to directly generate jailbreak suffixes for given harmful instructions. They are: **AmpleGCG** [11] and **AdvPrompter** [16].

Evaluation metrics. For the attack performance evaluation, we report the **Attack Success Rate (ASR)**, which is defined as the percentage of successful jailbreak attacks on the safety dataset. Specifically, for each sample from the safety dataset, we perform a jailbreak attack to synthesize a jailbreak suffix and use it to induce the targeted LLMs to generate 10 responses. An LLM-based judger [2] from Mazeika et al. [14] is then used to evaluate whether a response contains harmful information or not. An attack is determined to be successful if any of the 10 responses is determined to be harmful. A high ASR indicates a strong jailbreak effectiveness.

For the attack efficiency evaluation, we adopt two metrics, which are: (1) **Memory consumption in Gigabyte (GB)**, which measures the peak total GPU memory usage. (2) **Execution time in minutes**, which is the wall-clock time used for synthesizing all jailbreak suffixes for all 50 samples from the dataset. For model-based attack, this is the time used to generate all the training data using the heuristic method.

A low memory consumption and a short execution time indicate a high efficiency for the evaluated suffix jailbreak attack.

Baseline acceleration methods. Our experiments evaluate the effectiveness of our proposed *PSKV* by comparing it against two alternatives: a baseline implementation where **no specific acceleration method is deployed** (which we denote as **None**), and an implementation using a **Standard KV Cache**. The implementation details of these methods are as follows:

- **None:** This baseline represents an unaccelerated, albeit batched, approach. In each iteration, it constructs $K \cdot q$ full candidate prompts by concatenating the prefix $x^{(h)}$ with each suffix. It then performs a single, large batched forward pass on these $K \cdot q$ sequences. This method does not reuse any computations and thus redundantly calculates the prefix KV vectors for every candidate in every iteration.
- **Standard KV Cache:** This baseline implements the intuitive optimization of pre-computing the prefix cache. It first performs a single forward pass on the prefix $x^{(h)}$ to obtain its KV cache. Then, to evaluate the $K \cdot q$ suffix candidates in a parallel batch, it physically replicates this prefix cache $K \cdot q$ times in GPU memory (e.g., via `torch.Tensor.expand`) to match the batch dimension of the suffixes. While this saves redundant computation, it incurs a significant upfront memory cost for storing the duplicated cache. For experiments involving a batch of B prompts, this duplication is performed for each prompt, resulting in a total of $B \cdot K \cdot q$ prefix cache copies in memory.

In contrast to these baselines, our *PSKV* (detailed in Section 4) is designed to achieve the computational savings of the standard cache while avoiding its prohibitive memory costs.

Implementation details. We re-implemented all six evaluated suffix jailbreak attacks by ourselves to make them compatible with our PSKV method. For the length of synthesized jailbreak suffixes, we fix it to 20 in every experiment. Please refer to Appendix A for detailed settings of all other hyperparameters of all attacks. All of the methods were benchmarked on 2 80GB A100 GPUs.

We use PSKV to accelerate (1) the suffix synthesis process of optimization-based suffix attacks and (2) the suffix synthesis during the attack model training data generation process of model-based suffix attacks.

5.2 Results Analysis

In this section, we study the effectiveness of PSKV in accelerating suffix jailbreak attacks from different aspects.

Prefix-shared Cache does not affect the attack performance of accelerated suffix jailbreak attacks. According to the design of PSKV described in Section 4, PSKV is a performance-neutral acceleration method that does not affect the attack performance of accelerated suffix jailbreak attacks. This is because PSKV only removes redundant KV vectors but leaves the necessary KV vectors intact during suffix jailbreaking. Nevertheless, here we conduct experiments to empirically justify this.We collect and present the ASRs of the jailbreak attacks that without any accelerations and those that adopt PSKV in Table 2. From the table, we observe minor fluctuations in ASR between the baseline and PSKV-accelerated attacks. These variations are not caused by PSKV altering the core attack logic, but are attributable to the inherent stochasticity in the end-to-end evaluation pipeline. This includes randomness both in the heuristic search algorithms used for suffix synthesis and in the sampling-based generation of multiple responses for ASR calculation. Overall, the attack performance remains broadly consistent

[1] https://huggingface.co/models
[2] https://huggingface.co/cais/HarmBench-Llama-2-13b-cls

Table 2: Attack Success Rates (ASRs) (%) of suffix jailbreak attacks with/without PSKV. A high ASR indicate a strong jailbreak attack performance. Results show that our PSKV is a lossless acceleration method that would not affect the attack performance of accelerated suffix jailbreak attacks.

Model	Acceleration	GCG	GCQ	Zhu's AutoDAN	BEAST	AmpleGCG	AdvPrompter
Vicuna-7B	None	96	86	54	92	94	98
	PSKV (Ours)	94	96	56	100	96	98
Llama-2-7B	None	70	18	16	36	96	96
	PSKV (Ours)	56	16	28	46	88	96
Llama-3-8B	None	58	24	32	38	72	72
	PSKV (Ours)	62	26	30	54	76	82
Mistral-7B	None	80	72	86	86	86	94
	PSKV (Ours)	78	76	84	86	92	92
Qwen-2.5-7B	None	88	84	4	90	84	96
	PSKV (Ours)	88	92	6	86	88	94

Table 3: Time usage (in Minutes) under different acceleration methods of: (1) synthesizing jailbreak suffixes when performing optimization-based attacks, and (2) synthesizing jailbreak suffix training samples for training the suffix generator in model-based attacks. We also calculate the speed-up rate of each attack when compared with the one that does not adopt any acceleration method, shown as "(× [rate])". In all analyzed cases, our Prefix-shared KV Cache method achieves the lowest time usage as well as the highest speed-up rate.

Model	Acceleration	Optimization-based Attacks				Model-based Attacks	
		GCG	GCQ	Zhu's AutoDAN	BEAST	AmpleGCG	AdvPrompter
Vicuna-7B	None	258 (× 1.00)	247 (× 1.00)	116 (× 1.00)	39 (× 1.00)	384 (× 1.00)	468 (× 1.00)
	Standard KV Cache	257 (× 1.00)	248 (× 1.00)	114 (× 1.02)	30 (× 1.30)	371 (× 1.04)	473 (× 1.00)
	Prefix-shared Cache (Ours)	174 (× 1.48)	168 (× 1.47)	67 (× 1.73)	28 (× 1.39)	278 (× 1.38)	330 (× 1.42)
Llama-2-7B	None	252 (× 1.00)	240 (× 1.00)	113 (× 1.00)	39 (× 1.00)	355 (× 1.00)	450 (× 1.00)
	Standard KV Cache	251 (× 1.00)	241 (× 1.00)	112 (× 1.01)	31 (× 1.26)	353 (× 1.01)	470 (× 0.96)
	Prefix-shared Cache (Ours)	170 (× 1.48)	162 (× 1.48)	64 (× 1.77)	27 (× 1.44)	269 (× 1.32)	329 (× 1.37)
Llama-3-8B	None	249 (× 1.00)	236 (× 1.00)	111 (× 1.00)	38 (× 1.00)	344 (× 1.00)	450 (× 1.00)
	Standard KV Cache	248 (× 0.95)	237 (× 1.00)	109 (× 1.02)	38 (× 1.00)	335 (× 1.03)	450 (× 1.00)
	Prefix-shared Cache (Ours)	162 (× 1.54)	148 (× 1.59)	59 (× 1.88)	25 (× 1.52)	226 (× 1.52)	319 (× 1.41)
Mistral-7B	None	243 (× 1.00)	234 (× 1.00)	109 (× 1.00)	37 (× 1.00)	375 (× 1.00)	497 (× 1.00)
	Standard KV Cache	243 (× 1.00)	234 (× 1.00)	108 (× 1.01)	37 (× 1.00)	455 (× 0.82)	459 (× 1.08)
	Prefix-shared Cache (Ours)	155 (× 1.57)	148 (× 1.58)	58 (× 1.88)	25 (× 1.48)	277 (× 1.35)	324 (× 1.53)
Qwen2.5-7B	None	281 (× 1.00)	270 (× 1.00)	126 (× 1.00)	33 (× 1.00)	360 (× 1.00)	454 (× 1.00)
	Standard KV Cache	280 (× 1.00)	269 (× 1.00)	124 (× 1.02)	33 (× 1.00)	432 (× 0.83)	416 (× 1.09)
	Prefix-shared Cache (Ours)	161 (× 1.75)	151 (× 1.79)	58 (× 2.17)	24 (× 1.38)	247 (× 1.46)	255 (× 1.78)

across all cases, demonstrating that PSKV effectively accelerates the attacks without systematically harming their original effectiveness.

Prefix-shared Cache significantly accelerates the execution of suffix jailbreaking. We calculated the time usage of conducting suffix jailbreak attacks under different acceleration methods (or no acceleration) in Table 3. From the table, we find that, compared with KV cache-based acceleration or no acceleration, our Prefix-shared Cache achieves the lowest jailbreaking time usage in all analyzed cases and realizes a jailbreaking speed-up rate of around 1.5 in most experiments. Besides, in some cases (*e.g.*, "Zhu's AutoDAN vs. Llama3-8B" and "Zhu's AutoDAN vs. Qwen2.5-7B"), jailbreak attacks accelerated by the Prefix-shared Cache can even be nearly twice as fast as their no-acceleration counterparts. Furthermore, even in the worst cases (*e.g.*, "AdvPrompter vs. Llama2-7B" and "BEAST vs. Qwen2.5-7B"), Prefix-shared Cache can still speed up the attack by at least 37%. All these results suggest that our proposed Prefix-shared Cache method can effectively reduce calculation cost and thus accelerate suffix jailbreak attacks.

The memory consumption of suffix jailbreaking accelerated by Prefix-shared Cache is low. We also calculated the memory consumption of different suffix jailbreak attacks in Table 4, from which we have two observations. When compared with standard KV cache inference acceleration, our Prefix-shared Cache method consumes approximately 25% to 50% less memory in most analyzed

Table 4: Memory consumption (in GB) under different acceleration methods of: (1) synthesizing jailbreak suffixes when performing optimization-based attacks, and (2) synthesizing jailbreak suffix training samples for training the suffix generator in model-based attacks. Our Prefix-shared KV Cache method achieves the lowest memory consumption in most analyzed cases.

Model	Acceleration	Optimization-based Attacks				Model-based Attacks	
		GCG	GCQ	AutoDAN	BEAST	AmpleGCG	AdvPrompter
Vicuna-7B	None	30.02	37.26	36.73	82.92	55.08	41.26
	Standard KV Cache	53.59	81.80	70.71	88.09	66.50	53.02
	Prefix-shared Cache (Ours)	27.32	30.47	27.31	54.80	53.13	46.61
Llama-2-7B	None	29.72	36.83	36.30	81.74	37.82	33.71
	Standard KV Cache	52.87	80.58	69.70	88.26	56.61	54.11
	Prefix-shared Cache (Ours)	27.26	30.17	27.26	53.61	40.05	46.61
Llama-3-8B	None	34.47	54.14	50.00	85.96	66.47	80.75
	Standard KV Cache	37.69	60.42	52.56	96.02	67.80	95.58
	Prefix-shared Cache (Ours)	28.00	41.21	33.67	57.43	55.64	86.61
Mistral-7B	None	31.52	37.37	38.59	88.06	40.41	37.23
	Standard KV Cache	33.48	47.52	43.31	68.17	44.34	41.29
	Prefix-shared Cache (Ours)	24.78	27.52	25.12	54.61	37.81	48.95
Qwen2.5-7B	None	41.17	68.10	60.24	90.89	68.64	94.84
	Standard KV Cache	42.82	71.33	61.53	94.82	75.00	90.04
	Prefix-shared Cache (Ours)	29.12	44.41	34.58	63.41	62.64	83.42

Table 5: Scalability of iterative/search-based attacks with PSKV on HarmBench using a Llama-3-8B target. We report total time/memory and the amortized cost per candidate suffix.

Attack	Width	Total Cost		Cost/width		ASR (%)
		Time (min)	Mem. (GB)	Time (min)	Mem. (GB)	
GCG	16	54	23.81	3.37	1.49	50
	32	90	27.99	2.81	0.88	54
	64	162	27.99	2.53	0.44	62
GCQ	16	43	41.22	2.69	2.58	26
	32	77	41.21	2.41	1.29	26
	64	148	41.21	2.31	0.64	26
BEAST	(5, 5)	11	45.39	0.44	1.82	40
	(10, 10)	16	46.58	0.16	0.47	50
	(15, 15)	25	57.43	0.11	0.25	54

scenarios. When compared with vanilla jailbreak attacks that do not adopt any acceleration techniques, we find that our Prefix-shared Cache method maintains the same or lower level of memory consumption. Combined with all previous results, these observations demonstrate that Prefix-shared Cache can help to realize fast and effective suffix jailbreaking with little memory overhead. This also potentially enables one to perform stronger suffix jailbreaking with modest computing resources, as hyperparameters that make suffix attacks to be stronger usually requires more memory for calculations.

Memory amortization with search width. An important advantage of our expandable KV cache is its ability to amortize memory costs when search width increases. As shown in Table 5, when width grows, each prompt generates more candidate suffixes that share the same cached prefix through our expansion mechanism. While total memory consumption increases with more candidates, the memory cost per suffix actually decreases because the prefix cache is shared across all candidates rather than recomputed for each one.

However, this amortization effect is sublinear rather than proportional to the width increase. This is because beyond the PSKV, memory is also consumed by the model parameters, per-suffix activations, gradient buffers, and other intermediate tensors, all of which do not benefit from sharing across candidates. As width increases, only the prefix cache component is amortized while these other components scale linearly with the number of candidates, resulting in diminishing but consistent memory savings per suffix.

Table 5 confirms this trend across all methods. For instance, increasing width from 16 to 64 (a 4× increase) results in less than 4× increase in total memory, demonstrating effective cache sharing. The amortized memory per suffix consistently decreases with larger width across all attacks. This effect is particularly pronounced for BEAST with high parallelism (e.g., beam_size × search_width = 15 × 15 = 225 candidates), where shared prefix caching provides substantial memory savings compared to standard approaches that would recompute prefixes for each candidate.

6 Conclusion

The effectiveness of suffix jailbreak attacks is fundamentally tied to heuristic searches that require a vast number of resource-intensive LLM inferences, creating a significant barrier to efficient security evaluation. Our work is motivated by the key observation : for a single harmful prompt, numerous candidate suffixes are generated, yet standard methods create multiple redundant copies of the prompt's computational state, leading to massive memory waste. To address

this inefficiency, we propose the *Prefix-Shared KV Cache (PSKV)* framework. The core principle of PSKV is to maintain only a single, shared, and expandable KV cache for each harmful instruction prefix, which is then efficiently reused for all its corresponding suffix candidates. To enable this sharing mechanism at scale for batches of diverse prompts, we also introduce a suffix-centric alignment strategy. We empirically validated PSKV across six suffix attacks on five LLMs, demonstrating an average reduction of 40% in time and 30% in memory compared to cache-optimized baselines, while preserving the original attack success rate.

References

[1] Patrick Chao, Alexander Robey, Edgar Dobriban, Hamed Hassani, George J. Pappas, and Eric Wong. 2024. Jailbreaking Black Box Large Language Models in Twenty Queries. arXiv:2310.08419 [cs.LG] https://arxiv.org/abs/2310.08419

[2] Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing. 2023. Vicuna: An Open-Source Chatbot Impressing GPT-4 with 90%* ChatGPT Quality. LMSYS Org Blog. https://lmsys.org/blog/2023-03-31-vicuna/ Accessed: 2025-09-14.

[3] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. arXiv:2205.14135 [cs.LG] https://arxiv.org/abs/2205.14135

[4] Badhan Chandra Das, M. Hadi Amini, and Yanzhao Wu. 2024. Security and Privacy Challenges of Large Language Models: A Survey. arXiv:2402.00888 [cs.CL] https://arxiv.org/abs/2402.00888

[5] Badhan Chandra Das, M. Hadi Amini, and Yanzhao Wu. 2025. Security and Privacy Challenges of Large Language Models: A Survey. *ACM Comput. Surv.* 57, 6, Article 152 (Feb. 2025), 39 pages. doi:10.1145/3712001

[6] Hao Di, Tong He, Haishan Ye, Yinghui Huang, Xiangyu Chang, Guang Dai, and Ivor Tsang. 2025. ProAdvPrompter: A Two-Stage Journey to Effective Adversarial Prompting for LLMs. In *The Thirteenth International Conference on Learning Representations*.

[7] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The llama 3 herd of models. *arXiv e-prints* (2024), arXiv–2407.

[8] Jonathan Hayase, Ema Borevković, Nicholas Carlini, Florian Tramèr, and Milad Nasr. 2024. Query-based adversarial prompt generation. *Advances in Neural Information Processing Systems* 37 (2024), 128260–128279.

[9] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7B. arXiv:2310.06825 [cs.CL] https://arxiv.org/abs/2310.06825

[10] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)* (2023).

[11] Zeyi Liao and Huan Sun. 2024. Amplegcg: Learning a universal and transferable generative model of adversarial suffixes for jailbreaking both open and closed llms. *arXiv preprint arXiv:2404.07921* (2024).

[12] Frank Weizhen Liu and Chenhui Hu. 2024. Exploring Vulnerabilities and Protections in Large Language Models: A Survey. arXiv:2406.00240 [cs.LG] https://arxiv.org/abs/2406.00240

[13] Yanxu Mao, Tiehan Cui, Peipei Liu, Datao You, and Hongsong Zhu. 2025. From LLMs to MLLMs to Agents: A Survey of Emerging Paradigms in Jailbreak Attacks and Defenses within LLM Ecosystem. *arXiv preprint arXiv:2506.15170* (2025).

[14] Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhaee, Nathaniel Li, Steven Basart, Bo Li, David Forsyth, and Dan Hendrycks. 2024. HarmBench: A Standardized Evaluation Framework for Automated Red Teaming and Robust Refusal. arXiv:2402.04249 [cs.LG] https://arxiv.org/abs/2402.04249

[15] Anay Mehrotra, Manolis Zampetakis, Paul Kassianik, Blaine Nelson, Hyrum Anderson, Yaron Singer, and Amin Karbasi. 2024. Tree of Attacks: Jailbreaking Black-Box LLMs Automatically. arXiv:2312.02119 [cs.LG] https://arxiv.org/abs/2312.02119

[16] Anselm Paulus, Arman Zharmagambetov, Chuan Guo, Brandon Amos, and Yuandong Tian. 2024. Advprompter: Fast adaptive adversarial prompting for llms. *arXiv preprint arXiv:2404.16873* (2024).

[17] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. 2023. Efficiently scaling transformer inference. *Proceedings of machine learning and systems* 5 (2023), 606–624.

[18] Vinu Sankar Sadasivan, Shoumik Saha, Gaurang Sriramanan, Priyatham Kattakinda, Atoosa Chegini, and Soheil Feizi. 2024. Fast Adversarial Attacks on Language Models In One GPU Minute. In *International Conference on Machine Learning*.

[19] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).

[20] Alexander Wei, Nika Haghtalab, and Jacob Steinhardt. 2023. Jailbroken: How Does LLM Safety Training Fail?. In *Conference on Neural Information Processing Systems*.

[21] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).

[22] Yifan Yao, Jinhao Duan, Kaidi Xu, Yuanfang Cai, Zhibo Sun, and Yue Zhang. 2024. A survey on large language model (LLM) security and privacy: The Good, The Bad, and The Ugly. *High-Confidence Computing* 4, 2 (2024), 100211. doi:10.1016/j.hcc.2024.100211

[23] Lu Ye, Ze Tao, Yong Huang, and Yang Li. 2024. ChunkAttention: Efficient Self-Attention with Prefix-Aware KV Cache and Two-Phase Partition. *arXiv preprint arXiv:2402.15220* (2024).

[24] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. 2024. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems* 37 (2024), 62557–62583.

[25] Sicheng Zhu, Ruiyi Zhang, Bang An, Gang Wu, Joe Barrow, Zichao Wang, Furong Huang, Ani Nenkova, and Tong Sun. 2023. AutoDAN: Interpretable Gradient-Based Adversarial Attacks on Large Language Models. arXiv:2310.15140 [cs.CR] https://arxiv.org/abs/2310.15140

[26] Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J. Zico Kolter, and Matt Fredrikson. 2023. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043* (2023).

A Implementation Details of Jailbreak Attacks

Our experiments adopt six token-level jailbreak attacks, including four optimization-based methods: GCG [26], BEAST [18], GCQ [8], and Zhu's AutoDAN [25], as well as two model-based attacks: AmpleGCG [11] and AdvPrompter [16]. We re-implemented all attacks to apply PSKV and enable efficient batched operations. The hyper-parameters for each attack are configured as follows:

- **GCG:** Following Algorithm 1 in Zou et al. [26] and its official repository, we set: iterations = 500, top-k = 256, batch size (search width) = 64, prompt batch size = 16, and search width batch size = 16.
- **GCQ:** Following Algorithm 1 in Hayase et al. [8], we set: iterations = 500, top-k = 256, proxy batch size and query batch size = 15, buffer size (search width) = 64, and batch size for prompts and buffer = 16.
- **BEAST:** Following Algorithm 1 in Sadasivan et al. [18] and its official repository, we set: both beam-search parameters = 15, and beam-search batch size = 15.
- **Zhu's AutoDAN:** Following Algorithms 1 and 2 in Zhu et al. [25], we set: iterations = 3, objective weights = (10, 100), top-B = 256, and temperature = 2.
- **AmpleGCG:** Following Algorithm 1 in Liao and Sun [11] and its official repository, we set: steps = 500, batch size (search width) = 64, and prompt batch size = 16.
- **AdvPrompter:** Following Algorithm 1 in Paulus et al. [16] and its official repository, we set: epochs = 10, candidate set size = 256, prompt batch size = 8, temperature = 0.6, penalty parameter = 1.2, beam set size = 64, and beam number = 4.

B Proof: time and memory complex

B.1 Complexity Analysis for Single Prompt

In this section, we provide the derivations for the computational (C) and memory (M) complexities presented in Section 4.2. We let E be the number of synthesis iterations; N_p, N_s, N_t be the token lengths of the prefix, suffix, and target, respectively; and $K \cdot q$ be the number of candidates evaluated per iteration. Our analysis abstracts away model-specific constants (e.g., hidden dimension, number of layers) as they are common factors across all methods. The core assumptions are that the attention mechanism has a computational cost of $O(L^2)$ for a sequence of length L, and the corresponding KV cache has a memory footprint of $O(L)$.

B.1.1 Method without Acceleration.

Computational Complexity. This method recomputes the entire sequence of length $N_p + N_s + N_t$ for each of the $K \cdot q$ candidates in every one of the E iterations. The total complexity is the product of the number of operations and the cost per operation:

$$\begin{aligned} C_{\text{NoAccel}} &= E \cdot (K \cdot q) \cdot O((N_p + N_s + N_t)^2) \\ &= O(E \cdot (K \cdot q) \cdot (N_p + N_s + N_t)^2). \end{aligned} \tag{4}$$

Memory Complexity. The peak memory is determined by the batched inference step, which must hold the KV caches for all $K \cdot q$ candidates simultaneously. The memory for each cache is proportional to the full sequence length.

$$\begin{aligned} M_{\text{NoAccel}} &= (K \cdot q) \cdot O(N_p + N_s + N_t) \\ &= O((K \cdot q) \cdot (N_p + N_s + N_t)). \end{aligned} \tag{5}$$

B.1.2 Standard KV Cache.

Computational Complexity. This method involves a one-time prefix computation and an iterative evaluation step over E iterations.

- **Prefix Computation (One-time):** A single forward pass over the prefix of length N_p costs $O(N_p^2)$.
- **Iterative Evaluation (per epoch):** For each of the $N_s + N_t$ tokens of the suffix and target, attention is computed over a context that grows from N_p to $N_p + N_s + N_t - 1$. The average context length for these tokens is approximately $N_p + (N_s + N_t)/2$. The computational cost for each of the $K \cdot q$ candidates is thus roughly proportional to the number of new tokens times twice the average context length, giving $O((N_s + N_t) \cdot (2N_p + N_s + N_t))$.

The total complexity is the sum of the one-time cost and the total iterative cost:

$$\begin{aligned} C_{\text{Std}} &= O(N_p^2) + E \cdot (K \cdot q) \cdot O((N_s + N_t)(2N_p + N_s + N_t)) \\ &= O(N_p^2 + E \cdot (K \cdot q) \cdot (N_s + N_t)(2N_p + N_s + N_t)). \end{aligned} \tag{6}$$

Memory Complexity. To achieve parallel computation, the pre-computed prefix cache of length N_p must be physically duplicated $K \cdot q$ times. The total memory is the sum of these duplicated prefix caches and the caches for the suffix and target segments.

$$\begin{aligned} M_{\text{Std}} &= (K \cdot q) \cdot O(N_p) + (K \cdot q) \cdot O(N_s + N_t) \\ &= O((K \cdot q) \cdot (N_p + N_s + N_t)). \end{aligned} \tag{7}$$

B.1.3 PSKV (Ours).

Computational Complexity. The computational process of PSKV is identical to the standard KV cache approach, as it also performs a one-time prefix computation followed by iterative evaluation of the suffix and target segments.

$$C_{\text{Ours}} = C_{\text{Std}} = O(N_p^2 + E \cdot (K \cdot q) \cdot (N_s + N_t)(2N_p + N_s + N_t)). \tag{8}$$

Memory Complexity. The key advantage of PSKV is that it avoids the physical duplication of the prefix cache. It stores the prefix cache of length N_p only once, alongside the caches for the $K \cdot q$ suffix and target segments.

$$\begin{aligned} M_{\text{Ours}} &= O(N_p) + (K \cdot q) \cdot O(N_s + N_t) \\ &= O(N_p + (K \cdot q) \cdot (N_s + N_t)). \end{aligned} \tag{9}$$

B.2 Derivations for Batched Complexity Analysis

In this section, we provide the derivations for the complexities of batched suffix synthesis presented in Section 4.3. We let B be the number of prompts, E be the number of synthesis iterations, and $K \cdot q$ be the candidates per prompt per iteration. The total candidates per iteration is $B \cdot K \cdot q$. Let $N_{p,\max}, N_s, N_{t,\max}$ be the maximum token lengths of the aligned prefix, suffix, and target, and $N_{p,\text{avg}}, N_{t,\text{avg}}$ be the average lengths.

B.2.1 Method without Acceleration.

Computational Complexity. This method recomputes the full sequence for every candidate in every iteration. The total number of sequences processed is $E \cdot B \cdot K \cdot q$. The cost for each is quadratic to its length.

$$\begin{aligned} C_{\text{NoAccel}} &= E \cdot (B \cdot K \cdot q) \cdot O((N_{p,\text{avg}} + N_s + N_{t,\text{avg}})^2) \\ &= O(E \cdot (B \cdot K \cdot q) \cdot (N_{p,\text{avg}} + N_s + N_{t,\text{avg}})^2). \end{aligned} \tag{10}$$

Memory Complexity. The peak memory is determined by the batched inference step, which holds the KV caches for all $B \cdot K \cdot q$ candidates simultaneously.

$$\begin{aligned} M_{\text{NoAccel}} &= (B \cdot K \cdot q) \cdot O(N_{p,\text{avg}} + N_s + N_{t,\text{avg}}) \\ &= O((B \cdot K \cdot q) \cdot (N_{p,\text{avg}} + N_s + N_{t,\text{avg}})). \end{aligned} \tag{11}$$

B.2.2 Standard KV Cache.

Computational Complexity. This method pre-computes the cache for the B prefixes once, then iteratively evaluates the suffixes and targets.

- **Prefix Computation (One-time):** A batched pass over B prefixes of max length $N_{p,\max}$ costs $O(B \cdot N_{p,\max}^2)$.
- **Iterative Evaluation (per epoch):** For each of the $B \cdot K \cdot q$ candidates, processing the $N_s + N_{t,\max}$ new tokens costs approximately $O((N_s + N_{t,\max}) \cdot (2N_{p,\max} + N_s + N_{t,\max}))$.

The total complexity over E iterations is the sum of the one-time cost and the total iterative cost:

$$C_{\text{Std}} = O(B \cdot N_{p,\max}^2 + E \cdot (B \cdot K \cdot q) \cdot (N_s + N_{t,\max})(2N_{p,\max} + N_s + N_{t,\max})). \tag{12}$$

Memory Complexity. To process in a single batch, the B prefix caches must be duplicated $K \cdot q$ times each. The total memory is the sum of these duplicated prefix caches and the caches for the suffix/target segments.

$$\begin{aligned} M_{\text{Std}} &= (B \cdot K \cdot q) \cdot O(N_{p,\max}) + (B \cdot K \cdot q) \cdot O(N_s + N_{t,\max}) \\ &= O((B \cdot K \cdot q) \cdot (N_{p,\max} + N_s + N_{t,\max})). \end{aligned} \tag{13}$$

B.2.3 PSKV (Ours).

Computational Complexity. The computational process is identical to the standard KV cache approach.

$$C_{\text{Ours}} = C_{\text{Std}} = O(B \cdot N_{p,\max}^2 + E \cdot (B \cdot K \cdot q) \cdot (N_s + N_{t,\max})(2N_{p,\max} + N_s + N_{t,\max})). \tag{14}$$

Memory Complexity. PSKV avoids physical duplication. It stores one cache for each of the B prefixes, and the caches for the $B \cdot K \cdot q$ suffix/target segments.

$$\begin{aligned} M_{\text{Ours}} &= B \cdot O(N_{p,\max}) + (B \cdot K \cdot q) \cdot O(N_s + N_{t,\max}) \\ &= O(B \cdot N_{p,\max} + (B \cdot K \cdot q) \cdot (N_s + N_{t,\max})). \end{aligned} \tag{15}$$